AI Compiler Performance Engineering

Auto-Loops Books — ch18

Contents

1 IREE: MLIR-Native Runtime and Compilation **1**

1.1 IREE HW matrix 1

1.2 IREE arch passes 4

1.3 IREE codegen 7

1.4 IREE tuning 10

1.5 IREE case study 12

1.6 Key Takeaways 16

1.7 Conclusion 17

Bibliography **19**

Index **21**

Chapter 1

IREE: MLIR-Native Runtime and Compilation

IREE attacks the decode problem from a different angle than a kernel compiler: it compiles the model's *scheduling logic and its device execution together*, ahead of time, into one deployable artifact (IREE / iree-org contributors, 2024). Where a Python serving loop decides per-token what to launch at runtime, IREE bakes the host orchestration into virtual-machine bytecode at compile time—which is exactly the lever Chapter 1 identified, because at batch-1 the launch and dispatch path, not the matmul, sets ms/token (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Dense Llama-3.2-1B already averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). A compiler that schedules the whole program can collapse that orchestration the way captured CUDA Graphs do, but earlier and across more targets (Chen, 2026; NVIDIA, 2024). The catch is the same as every other surface in this Part: IREE will happily produce a correct binary for a target whose schedule still re-reads activations from DRAM every token. This chapter treats IREE as an MLIR-native compiler+runtime graded on bytes/token and launches/token, not as a build-system walkthrough (Goodfellow *et al.*, 2016).

1.1 IREE HW matrix

IREE's support **matrix** is the broadest in this Part: one MLIR input compiles to llvm-cpu, CUDA, ROCm/HIP, Vulkan/SPIR-V, and Metal, with AMD AIE still experimental (IREE / iree-org contributors, 2024). The Hardware Abstraction

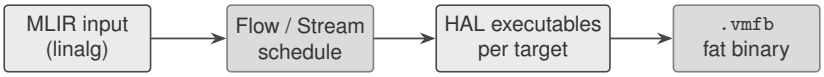


Figure 1.1: IREE lowers one MLIR input through whole-program scheduling into a single `.vmfb` that carries host instructions and per-target device kernels.

Layer (HAL) is what makes that breadth tractable—it is a Vulkan-inspired **target** abstraction, roughly 1:1 with IREE's HAL C API, that exposes buffers, command buffers, and dispatches as a common surface across device types (IREE / iree-org contributors, 2024).

The honest reading is that a wide build matrix is not a wide *decode* matrix. A green HAL **target** means the model lowers and runs there; it says nothing about whether the schedule keeps a decoder layer on-chip (Vellaisamy *et al.*, 2026; Chen, 2026). The hardware truth is unchanged across the matrix: a **gpu** is bound by shared-memory residency and tensor-core feeding, a **cpu** by cache blocking and NUMA, an **npu** like XDNA/AIE by static tile SRAM and DMA chains—and on the IREE matrix the NPU path is still experimental, so it is exactly where the build-vs-decode gap is widest (NVIDIA, 2022; AMD, 2024b,a).

What makes the breadth credible rather than aspirational is that IREE was designed to scale from the datacenter down to mobile and edge, where the same compiled artifact must run under tight memory and power budgets (IREE / iree-org contributors, 2024). The runtime is deliberately small and optional: a fully featured server can use the VM for dynamic deployment with multi-threaded dispatch, while an embedded target can bypass the runtime entirely and link the generated kernels directly (IREE / iree-org contributors, 2024). For a decode engineer that flexibility is a double-edged sword. The same model that serves on an H100 can be compiled for an edge accelerator, but the residency math is completely different at the two ends of the matrix: an edge SKU with megabytes of on-chip memory cannot hold the working set a datacenter GPU does, so a dispatch-region fusion that is a clear win on H100 can overflow on-chip storage and spill on the edge target.

Multi-hardware delta. The same `.vmfb` can carry kernels for several vendors, and IREE can probe the device at load time to pick one (IREE / iree-org contributors, 2024). But a Hopper GPU wants async-copy double buffering while an XDNA fabric wants a static DMA schedule, so the portable artifact still hides per-target residency decisions that decode telemetry will expose (SemiAnalysis, 2024; AMD, 2024a). YiRage treats this as a routing problem: it carries the Chapter 2 residency rules as checkable metadata and selects a backend whose machine model fits,

rather than trusting that a green matrix cell is decode-legal (Chen and YiRage Contributors, 2026).

1.1.1 Input paths into IREE

The input path determines how much scheduling structure the compiler can see. IREE consumes several front ends: TensorFlow and TFLite models convert into the stablehlo-based input dialects, PyTorch models arrive through a torch-MLIR path, and ONNX models convert through the ONNX-MLIR lowering (IREE / iree-org contributors, 2024; Chen, 2020). Each conversion has a granularity problem similar to the ones XLA and TVM face: operations that arrive as fused framework calls may stay opaque, while operations that arrive element-by-element give the flow stage the seams it needs to form dispatch regions (Davies *et al.*, 2025; Chen, 2026). For decode, the recommended input is a graph where one decoder layer is a recognizable unit with attention and MLP structure preserved, because the dispatch-region formation can then fuse inside the layer instead of re-deriving structure from flat operator lists (Dao *et al.*, 2022; IREE / iree-org contributors, 2024). Teams should inspect the converted module before compilation and treat poor structure as an export bug rather than an IREE limitation (Chen, 2020; Davies *et al.*, 2025).

1.1.2 VM and embedded runtime modes

IREE's runtime flexibility is a deployment decision with decode consequences. The full VM runtime provides dynamic features such as module loading, multi-threaded dispatch, and debugging hooks, while the embedded mode links compiled kernels directly for small footprints and lower per-call overhead (IREE / iree-org contributors, 2024). A batch-1 serving path that runs in the VM still benefits from compiled scheduling because the bytecode is generated ahead of time, but every dynamic runtime call has a cost that shows up when measured per token (Vellaisamy *et al.*, 2026; Chen, 2026). Embedded mode removes most of that cost at the price of compile-time specialization: every shape and target variant is a separate linked artifact (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). The serving team should measure both modes at the decode operating point and choose by p99, not by deployment convenience (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Neither mode repairs a poorly fused dispatch region; both only execute the schedule the compiler built (Dao *et al.*, 2022; Chen, 2026).

1.1.3 Multi-target artifact strategy

Because one `.vmfb` can carry specialized kernels for multiple targets, the artifact strategy shapes the build matrix in production (IREE / iree-org contributors, 2024). Shipping one fat binary that supports every GPU vendor simplifies rollout but multiplies binary size and compile time; shipping per-SKU artifacts optimizes size and startup at the cost of more build jobs and a more complex release matrix (Davies *et al.*, 2025; Chen, 2026). For decode fleets with mixed hardware, the per-SKU approach usually wins because each target needs its own tuned configuration and residency proof anyway (NVIDIA, 2022; AMD, 2024a). The artifact manifest should record the targets, compiler version, input model hash, and decode benchmark for each binary so a rollout is reproducible (Chen, 2020; Vellaisamy *et al.*, 2026). This is the IREE instance of the rule that the deployed compiler output is a first-class artifact, not a build byproduct (Davies *et al.*, 2025; Chen, 2026).

1.1.4 Server, mobile, and edge decode contracts

IREE's breadth across server GPUs and mobile accelerators means the same model source serves multiple decode contracts, and the contracts differ in more than speed (IREE / iree-org contributors, 2024; Chen, 2026). A server contract can assume a large on-chip working set, deep async pipelines, and a fast host; an edge contract must fit the working set into smaller SRAM or cache, tolerate power limits, and pay closer attention to binary size and startup time (IREE / iree-org contributors, 2024; Goodfellow *et al.*, 2016). Dispatch-region fusion that is safe on a datacenter GPU can overflow the smaller on-chip storage of an edge target, so each contract needs its own fusion and codegen configuration (AMD, 2024b; Chen, 2026). The decode benchmark for an edge contract should include memory and power envelopes, not just latency, because a schedule that wins on ms/token by using more memory may fail the device budget (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). IREE's matrix is broad enough to support all three contracts, and exactly broad enough to make per-contract qualification non-optional (IREE / iree-org contributors, 2024; Chen, 2020).

1.2 IREE arch passes

IREE's compilation is a sequence of MLIR **dialect**s, and the **pass** pipeline is where decode goodput is decided long before any device code is generated (IREE / iree-org

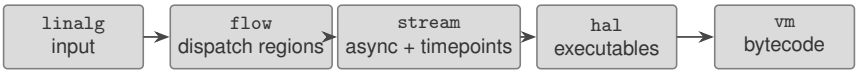


Figure 1.2: IREE's dialect pipeline: `flow` forms dispatch regions, `stream` schedules them asynchronously with timepoints, `hal` translates per target, `vm` emits host bytecode.

Table 1.1: IREE's ahead-of-time compilation phases; `flow` fusion sets bytes/token and `stream` scheduling sets launches/token (IREE / iree-org contributors, 2024).

Phase	What it does
Input	Lower to core dialects (`linalg`, etc.)
Flow	Form and fuse dispatch regions; partition
Stream	Async schedule with timepoints; encode resources
HAL	Per-target executable translation (codegen)
VM	Emit host scheduling bytecode

contributors, 2024). After the input **dialect** (linalg and friends), dispatch-creation fuses operations and forms *dispatch regions*—the unit of work that becomes one kernel; the `flow` dialect models data flow and partitioning; the `stream` dialect **lower**s tensors into opaque resources and schedules the dispatches asynchronously using *timepoints* so independent work overlaps; the `hal` dialect translates each executable per target; and the `vm` dialect emits the host bytecode (IREE / iree-org contributors, 2024).

Two of these passes are the ones a decode engineer must watch. Dispatch-region formation is the **optim**ization that sets bytes/token: if it fuses a decoder layer's element-wise and reduction work into one region, intermediates stay on-chip; if it leaves them separate, each boundary is a DRAM round trip (Vellaisamy *et al.*, 2026; Chen, 2026). Stream scheduling is the one that sets launches/token: by sequencing dispatches with timepoints instead of a host-side wait per kernel, it removes exactly the per-launch orchestration that dominates batch-1 (IREE / iree-org contributors, 2024; NVIDIA, 2024).

What to inspect before merge. IREE can dump the executable for each dispatch region (`-iree-hal-dump-executable-*`), so diff which operators were fused into a region and which spilled out; an aggressive-fusion flag exists, but more fusion is only a win until register or shared-memory pressure forces a spill (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). A regression here is silent: the binary is correct, only bytes/token moved (Goodfellow *et al.*, 2016).

1.2.1 Dispatch-region formation as fusion

The flow stage's central decision is where to cut the graph into dispatch regions, and the cut determines every downstream counter (IREE / iree-org contributors, 2024; Chen, 2026). A region that spans an elementwise chain, a reduction, and its consumer keeps the intermediate on-chip; a cut at any of those boundaries creates a kernel boundary and an HBM round trip (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). IREE uses an aggressive fusion policy by default, but legality still depends on shapes, layouts, and the target's on-chip budget (IREE / iree-org contributors, 2024; Chen, 2020). For decode, the review question is how many dispatch regions one decoder layer produced and which operators each region contains (Davies *et al.*, 2025; Chen, 2026). If the layer fragments into more regions than the intended fusion design, the fix is upstream in the input structure or fusion flags, not in the codegen (Chen, 2020; Davies *et al.*, 2025).

1.2.2 Stream scheduling and timepoints

The stream stage turns a fused dispatch list into an asynchronous schedule by attaching timepoints that express dependencies instead of inserting a host wait after each dispatch (IREE / iree-org contributors, 2024; NVIDIA, 2024). That design removes the per-kernel host synchronization that makes eager batch-1 decode expensive, because the device can run dependent work in order without the host round-tripping between dispatches (Vellaisamy *et al.*, 2026; Chen, 2026). Decode graphs are mostly sequential, so the timepoint win is modest compared with a wide parallel graph, but the removal of per-dispatch host waits is exactly the launches/token lever this book tracks (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The stream stage also decides how independent work overlaps, which matters for fused regions that split into compute and copy phases (IREE / iree-org contributors, 2024; SemiAnalysis, 2024). When profiling shows gaps between dispatches, inspect the stream schedule for unexpected serialization before blaming the kernels (Chen, 2026; Davies *et al.*, 2025).

1.2.3 VM bytecode and host control flow

The VM stage is what makes IREE different from a kernel library: the host side of the model becomes bytecode that encodes the dispatch sequence, buffer handling, and control flow (IREE / iree-org contributors, 2024). Decode loops, bucket selection, and per-token calls can be represented in that bytecode, which is why a compiled IREE program can replace a Python serving loop for the steady-state

path (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Control flow that stays host-side and dynamic, such as sampling and MoE routing, should be kept outside the compiled region and called explicitly so the VM does not try to encode logic that changes per request (Kwon *et al.*, 2023; Liu *et al.*, 2024). The boundary between compiled bytecode and dynamic host logic is a design decision, and the telemetry should attribute latency to whichever side actually spent it (Chen, 2026, 2020).

1.2.4 Resource encoding and buffer reuse

Between flow and stream, tensors become opaque resources with lifetimes the scheduler can manage (IREE / iree-org contributors, 2024; Chen, 2020). Resource encoding is where IREE plans buffer reuse across the whole module, and reuse is what lets a decoder layer stream a small working set instead of allocating fresh storage per operator (Chen, 2026; IREE / iree-org contributors, 2024). When encoding chooses a new resource for every intermediate, the runtime footprint grows and cache behavior degrades even if fusion succeeded (Davies *et al.*, 2025; Chen, 2026). The compile report should show resource counts and reuse decisions for the decode layer so reviewers can verify that intermediates are not accumulating per region (Chen, 2020; Vellaisamy *et al.*, 2026). Buffer reuse is the silent half of residency: fusion keeps values on-chip, and resource encoding keeps the off-chip footprint from growing (Chen, 2026; IREE / iree-org contributors, 2024).

1.3 IREE codegen

IREE's **codegen** runs per dispatch region during the HAL executable-translation stage, and each **backend emit**s a different low-level form: PTX for CUDA, HSACO for ROCm/HIP, SPIR-V for Vulkan, and native code for the CPU (IREE / iree-org contributors, 2024). Because compilation is ahead-of-time, the **platform** mix is resolved on the build host and packaged into the FlatBuffers `.vmfb` fat binary, which carries host VM instructions alongside the device kernels so the runtime can dispatch without a JIT (IREE / iree-org contributors, 2024).

The decode-relevant point is that IREE's runtime is not a Python loop; the VM replays a compiled schedule, so the host orchestration cost is paid at compile time rather than per token (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). This is structurally the same win that capturing a launch graph buys on a GPU—the Memory-Floor study recovered roughly 1.26× on an H100 decode cell purely by removing host overhead—except IREE bakes it into the deployable artifact and extends it to CPU and Vulkan targets (Chen, 2026; NVIDIA, 2024).

What IREE cannot do is rescue a poorly fused dispatch region: if codegen emits a layer as many small kernels because fusion failed upstream, no amount of clean scheduling removes the HBM traffic those kernels create (Dao *et al.*, 2022).

The fat-binary format is itself a decode-relevant design choice. Because the kernels are committed at build time, IREE can ship several specialized variants of the same dispatch region—aligned and unaligned shapes, large and small matmuls—and let the runtime probe the device and select one without a JIT step (IREE / iree-org contributors, 2024). For batch-1 decode that avoids the warmup and recompilation stalls a JIT stack pays on the first request after a shape change, which is precisely the moment an interactive session is most latency-sensitive. The cost is binary size and a build matrix that grows with every target and specialization, so a serving team trades disk and compile time for predictable per-token latency—usually the right trade for an interactive product, the wrong one for a one-off batch job.

Multi-hardware delta. The HAL maps the same dispatch onto workgroups that become GPU thread blocks or CPU threads, and subgroups that become GPU SIMT lanes or CPU SIMD (IREE / iree-org contributors, 2024). The abstraction is genuine, but the best tile shape and async depth still differ per target, so one dispatch region's codegen is not optimal everywhere (SemiAnalysis, 2024; NVIDIA, 2022).

1.3.1 Translation strategies per target

The HAL executable stage chooses a translation strategy for each dispatch region based on the target and the operator mix in the region (IREE / iree-org contributors, 2024; Chen, 2020). A region that is mostly GEMMs may translate through a vendor library call or a specialized codegen path, while an elementwise chain translates through the generic vector pipeline (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). The strategy choice is where per-target tuning lives: the CPU path wants vector widths and cache blocking, the CUDA path wants shared-memory and MMA-friendly codegen, and the Vulkan path wants SPIR-V shader workgroup sizes that fit the driver's best configuration (IREE / iree-org contributors, 2024; SemiAnalysis, 2024). For decode, the report should show which strategy each dispatch region used, because a GEMM region that took the generic path instead of a vendor call is usually a missed configuration rather than a compiler deficiency (Chen, 2026; Vellaisamy *et al.*, 2026). Teams should re-run strategy selection at the decode shape and compare the report against the prefill report, since strategy

choices that win on large shapes can be wrong for thin batch-1 work (Kwon *et al.*, 2023; Davies *et al.*, 2025).

1.3.2 Variant specialization and binary growth

Specialized variants trade binary size and compile time for predictable runtime behavior (IREE / iree-org contributors, 2024; Chen, 2026). Each aligned and unaligned shape pair, each matmul size, and each target can produce its own executable variant inside the fat binary, and the runtime selects one at load or dispatch time (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). For a decode server with a small bucket set, that specialization is exactly right: the compiler emits one good variant per bucket instead of forcing one general kernel to serve all shapes (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). The cost is a build and disk footprint that must be managed in the release process (Chen, 2020, 2026). Teams should define the specialization set from the serving bucket list and prune variants that no request ever uses, because dead variants still cost compile time and confuse binary triage (Davies *et al.*, 2025; Chen, 2026).

1.3.3 Device probing and dispatch

IREE's runtime can probe device capabilities and select among variants without a JIT step, which is the advantage the fat binary buys over a runtime compiler (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). Probe logic should be deterministic and logged: which device, which variant, and which benchmark produced the selection (Davies *et al.*, 2025; Chen, 2026). When a fleet mixes GPUs, the probe result explains why the same binary performs differently and points to the per-SKU variant set as the tuning surface (NVIDIA, 2022; AMD, 2024a). A probe that silently falls back to a generic variant is a p99 risk, so the runtime should expose the selected variant in telemetry (Kwon *et al.*, 2023; Chen, 2026). This turns device diversity from an incident source into a managed configuration dimension (Chen, 2020; Davies *et al.*, 2025).

1.3.4 Measuring dispatch overhead

Even with VM bytecode and stream timepoints, dispatch overhead does not vanish; it shrinks to the compiled cost of moving work through the HAL (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). That cost should be measured by comparing the device kernel time against the wall time of a token step at batch-1 (Davies *et al.*, 2025; Chen, 2026). If device time accounts for nearly all wall time,

the schedule is doing its job; if a gap remains, the gap is runtime dispatch, variant selection, or host-call overhead inside the VM (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). Reducing the gap may mean moving to embedded mode, tightening the VM module, or moving per-token control flow out of the compiled region (Chen, 2020; Vellaisamy *et al.*, 2026). The measurement belongs in the compile-time benchmark harness so every configuration change reports the overhead fraction alongside ms/token (Chen, 2026; IREE / iree-org contributors, 2024). This fraction is the IREE-specific view of launches/token: not how many launches, but how much each remaining launch costs through the runtime (NVIDIA, 2024; Chen, 2026).

1.4 IREE tuning

IREE **tun**ing happens at compile time through target configuration: the executable-configurations stage selects a translation strategy per dispatch region, and flags steer dispatch-region fusion and per-target codegen (IREE / iree-org contributors, 2024). Because the schedule is compiled, **profile**-first discipline shifts earlier than with a JIT stack—you compile a candidate, run it, and read where time went before changing flags, rather than letting a runtime autotuner converge.

There is a second discipline that whole-program compilation rewards: compile-time visibility. Because every dispatch region and its scheduling are decided before deployment, the compiler can report exactly which operators fused, how many regions a layer became, and how they are sequenced—information a runtime-dispatched stack only learns by profiling the live system. A decode team should treat that report as the first artifact to read after a build, comparing the dispatch-region count per layer against expectation before ever launching the model. When the region count is higher than the number of fused units you designed for, fusion failed somewhere upstream, and the fix belongs in the compilation flags, not in a runtime workaround.

The **pitfall** is benchmarking the wrong shape and the wrong layer. Prefill compiles into large, well-fused dispatch regions that look excellent in isolation; decode is dominated by the thin, latency-bound per-token path where dispatch granularity and host scheduling matter more than peak occupancy (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Compile and profile at the batch-1 decode operating point, decompose end-to-end ms/token into bytes/token and launches/token, and only then adjust fusion and codegen flags (Davies *et al.*, 2025; Chen, 2026). The right configuration is **hardware**-specific: the CPU path wants cache-blocked dispatch regions, the CUDA path wants fused kernels plus async copy, and the

experimental AIE path wants static tiling—so the tuned configuration for one HAL target does not transfer to another (AMD, 2024b; Goodfellow *et al.*, 2016).

1.4.1 Compile-time benchmarking workflow

Because IREE schedules at compile time, the tuning loop is: compile a candidate configuration, benchmark the artifact at the decode operating point, inspect the dispatch and strategy reports, adjust flags, and recompile (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). The workflow should be scripted end to end so every flag change produces an artifact with the same manifest fields and the same benchmark harness (Chen, 2020, 2026). The benchmark must measure the full token step through the VM or embedded runtime, not individual dispatch regions, because the schedule between regions is part of what is being tuned (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Each candidate should report dispatch-region count per layer, strategy selections, and bytes/token alongside ms/token, so the winning configuration is chosen for a reason that survives review (Chen, 2026; Davies *et al.*, 2025). This workflow replaces runtime autotuning with build-time search, which is more expensive per candidate but gives a deterministic deployment (IREE / iree-org contributors, 2024; Chen, 2020).

1.4.2 Fusion flags and their limits

Aggressive-fusion flags widen dispatch regions, but the limit is always the target's on-chip budget (IREE / iree-org contributors, 2024; Chen, 2026). A region that fuses an entire decoder layer may exceed registers, shared memory, or tile SRAM on the actual target and spill during codegen or execution (NVIDIA, 2022; AMD, 2024a; Dao *et al.*, 2022). The correct level is not the maximum fusion but the fusion that keeps occupancy healthy and intermediates resident (Vellaisamy *et al.*, 2026; Chen, 2026). For decode, fusion decisions should be made per dispatch-region structure: attention carriers and their consumers fuse well, while routing and sampling logic should stay outside regions (Dao *et al.*, 2022; Liu *et al.*, 2024). The executable dumps show whether a requested fusion survived or was split by legality, and that report is the tuning signal (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). Teams that assume a fusion flag is a promise rather than a request will be surprised by spills that the report would have predicted (Chen, 2026, 2020).

1.4.3 Fingerprinting compiled artifacts

Whole-program compilation makes artifact fingerprinting both easier and more necessary: easier because the artifact captures the schedule, more necessary because a small flag or compiler change can alter every dispatch region (IREE / iree-org contributors, 2024; Chen, 2020). The fingerprint should include the IREE version, the input module hash, the target configuration, the fusion and codegen flags, and the benchmark results of the artifact (Chen, 2026; Davies *et al.*, 2025). Release manifests should store the fingerprint beside the `.vmfb` so a rollout can be bisected to the exact configuration (Vellaisamy *et al.*, 2026; Chen, 2020). When a decode regression appears after an upgrade, the fix is to diff fingerprints and re-run the compile-time workflow, not to tune at runtime (Chen, 2026; Davies *et al.*, 2025). This discipline is the deployment half of IREE's design: compiled determinism only helps if the compilation itself is recorded (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026).

1.4.4 Quantization and mixed precision at compile time

Whole-program compilation makes precision decisions visible at the module level instead of scattered across kernel wrappers (IREE / iree-org contributors, 2024; Chen, 2020). Quantized weights, FP16 or BF16 activations, and per-tensor scales flow through the input dialects into dispatch regions, where the chosen precision determines bytes moved per token (Chen, 2026; Davies *et al.*, 2025). Mixed-precision compile must respect numerical boundaries: softmax carriers and accumulators need higher precision even when the surrounding matmuls run in reduced precision (Dao *et al.*, 2022; Chen, 2026). The compile report should state the precision of every dispatch region so a reviewer can verify that reduced precision did not silently creep into a numerically sensitive seam (Chen, 2020; Davies *et al.*, 2025). Quantization is therefore a compile-time residency decision in IREE: it changes bytes/token before fusion ever runs, and the two must be tuned together rather than separately (IREE / iree-org contributors, 2024; Chen, 2026).

1.5 IREE case study

Take a single decoder layer compiled through IREE as the **case**, and **benchmark** it at the decode operating point on more than one HAL target (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). The reading matches the book's thesis: a layer that compiles into one fused dispatch region with stream-scheduled launches

keeps activations on-chip and pays host orchestration once; a layer that fragments into many dispatch regions re-reads activations from DRAM and pays a launch per region every token (Chen, 2026; Dao *et al.*, 2022). The arithmetic is the same as for any fusion: at hidden size 4096 in BF16 each materialized activation vector is 8 KB, so a handful of avoided round trips per layer is tens of kilobytes of HBM traffic removed per token before the KV cache is even read (Vellaisamy *et al.*, 2026). Across the full depth of a model that avoided traffic accumulates into a meaningful fraction of the per-token memory budget, which is why the dispatch-region count, not the peak throughput of any single region, is the number that tracks decode latency. The same compiled layer that fragments into a dozen regions will read and rewrite those vectors a dozen times, and the runtime schedule cannot recover bytes the compiler chose to spill.

The **cross-hardware** conclusion is where IREE's breadth helps and where it stops. Because IREE keeps the same dispatch-region formation across targets, a layer fused well on the CPU path is usually fused on the CUDA path too, which makes A/B comparison and bring-up faster (IREE / iree-org contributors, 2024). But the experimental AIE/NPU path is exactly where a green build hides an unfinished decode story, and a Vulkan target on a phone has different residency limits than an H100 (AMD, 2024a; NVIDIA, 2022). **YiRage** uses IREE-style MLIR lowering as one path while enforcing the Chapter 2 residency rules as checkable IR metadata, so a dispatch region that would spill is rejected at compile time and non-GPU work is routed to a backend whose machine model fits (Chen and YiRage Contributors, 2026).

1.5.1 Setting the region-count target

The case study becomes measurable when the team states a dispatch-region target per decoder layer before compiling (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). A reasonable first target for a fused layer is a small number of regions: one for the attention core that respects online-softmax carriers, one or two for the projections and MLP if the on-chip budget forces a split, and none for trivial elementwise chains that should have fused away (Dao *et al.*, 2022; Chen, 2026). The compiler report should be checked against that target after every build, and a higher count is a finding, not an accepted default (Davies *et al.*, 2025; Chen, 2020). Region count is the compile-time proxy for bytes/token because each extra region is a potential HBM boundary (Chen, 2026; Vellaisamy *et al.*, 2026). When the count is right, the benchmark verifies the proxy; when the count drifts, the benchmark explains the regression (IREE / iree-org contributors, 2024; Davies *et al.*, 2025).

1.5.2 Reading the VM schedule

Beyond region count, the compiled VM schedule shows how regions are sequenced and where host-side waits remain (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). The schedule should read like a dependency graph with timepoints, not a list of kernels with a wait after each one (IREE / iree-org contributors, 2024; NVIDIA, 2024). Gaps between dependent regions in a profile are expected only when a real dependency exists; gaps between independent regions indicate scheduling that serialized work unnecessarily (SemiAnalysis, 2024; Chen, 2026). The decode case study should therefore inspect both the VM-level schedule and the device timeline, because the former explains the latter (Davies *et al.*, 2025; Chen, 2020). A compiled program whose schedule contains per-region host waits has lost the structural advantage IREE promises, and the fix is in the stream stage configuration (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026).

1.5.3 Failure autopsies for IREE decode

Three signatures recur. A correct binary with high bytes/token usually means dispatch-region fusion stopped early, visible as a high region count in the compile report (Chen, 2026; IREE / iree-org contributors, 2024). A correct binary with high ms/token but healthy region count usually means stream scheduling serialized work or the runtime selected a generic variant instead of the specialized one (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A regression that appears only after an IREE upgrade usually traces to changed fusion or translation defaults, recoverable by diffing the compile reports between versions (Chen, 2020, 2026). Each autopsy starts from the artifact fingerprint and compile report, which makes the diagnosis a diff instead of a re-derivation (Davies *et al.*, 2025; IREE / iree-org contributors, 2024). Teams that archive reports with binaries turn every recurrence into a ten-minute check (Chen, 2026; Vellaisamy *et al.*, 2026).

1.5.4 Integrating the compiled module into serving

The compiled `.vmfb` still needs a serving integration around it: tokenization, sampling, KV-cache management, and request scheduling live in the application, while the steady-state decoder path runs in the compiled module (Kwon *et al.*, 2023; IREE / iree-org contributors, 2024). The integration should call the module through the runtime API with preallocated buffers and avoid per-token allocation, because allocation on the hot path reintroduces the host cost the compiler removed (Davies *et al.*, 2025; Chen, 2026). KV-cache state that changes shape per token

must be handled by the compiled bucket contract: either the module accepts the current KV pointer and length as arguments, or the application switches modules at bucket boundaries (Chen, 2020; IREE / iree-org contributors, 2024). The telemetry boundary should attribute time to the module call, the runtime, and the application logic separately so regressions are attributable (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). A clean integration is the difference between IREE's compiled schedule being the serving path and being one kernel among many in an eager host (Chen, 2026; Davies *et al.*, 2025).

1.5.5 Multi-model packaging and load

Production serving often compiles several models or several buckets of one model into the same deployment, which makes module load and memory sharing a design question (Kwon *et al.*, 2023; IREE / iree-org contributors, 2024). Loading every variant eagerly wastes memory; loading lazily pays first-token latency on cache misses (Davies *et al.*, 2025; Chen, 2026). A serving team should define the resident module set from traffic data and preload the hot set at startup (Chen, 2020; Vellaisamy *et al.*, 2026). The runtime can share one HAL device context across modules, which reduces per-module overhead, but the sharing must be verified under concurrent dispatch (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). Release testing should include the load path: startup time, memory before and after load, and first-token latency per module (Chen, 2026; IREE / iree-org contributors, 2024). Multi-model packaging turns IREE's compile-time determinism into a fleet-level property rather than a per-binary property (Kwon *et al.*, 2023; Chen, 2020).

1.5.6 Working example checks

Before shipping an IREE decode artifact, run three working checks against the compiled output: the dispatch report matches the region target per layer, the VM schedule contains no per-region host waits, and the benchmark shows device time close to wall time at batch-1 (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026; Chen, 2026). Each check is fast once the workflow is scripted, and each catches a different failure class: fusion loss, scheduling serialization, and runtime overhead (Davies *et al.*, 2025; Chen, 2020). The checks should run on every target in the artifact because a configuration that passes on CUDA can fail on Vulkan or CPU (IREE / iree-org contributors, 2024; AMD, 2024a). Teams that automate the checks get release candidates that are already explained; teams that skip them get binaries whose behavior is discovered by customers (Chen, 2026; Davies *et al.*,

2025).

1.5.7 Reading target deltas

When the same decode module ships on multiple targets, the target delta report should compare region counts, strategy selections, and bytes/token so differences are understood rather than discovered (IREE / iree-org contributors, 2024; Chen, 2026). A target that fuses less is not wrong if its on-chip budget is smaller; it needs its own region target and its own benchmark (AMD, 2024b; Chen, 2020). A target whose dispatch overhead is higher needs a different runtime mode or variant selection (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). The delta report turns IREE's breadth from a support promise into a measurable matrix of decode contracts (Vellaisamy *et al.*, 2026; Chen, 2026). Without it, a multi-target rollout treats every difference as an incident instead of a configuration (Davies *et al.*, 2025; IREE / iree-org contributors, 2024). The report belongs in the release manifest so that a later IREE upgrade can be evaluated target by target instead of by a single headline latency number, and so that the residency contract for each HAL target stays visible to the whole serving team (IREE / iree-org contributors, 2024; Chen, 2026). Each manifest cell should name the runtime mode, the variant set, and the measured dispatch-overhead fraction, making the delta auditable across releases and across the engineers who own those targets (IREE / iree-org contributors, 2024; Davies *et al.*, 2025).

1.6 Key Takeaways

- **IREE compiles scheduling and execution together.** Host orchestration is baked into VM bytecode ahead of time, so the per-token launch path is set at compile time, not by a runtime loop (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). That is the structural fix for the launches/token problem decode hits (Davies *et al.*, 2025).

- **Dispatch-region fusion sets bytes/token.** Whether the `flow` stage fuses a decoder layer into one region decides how many DRAM round trips survive; dump and diff the executables before trusting a build (IREE / iree-org contributors, 2024; Chen, 2026). A correct binary can still be a slow token (Goodfellow *et al.*, 2016).

- **Stream scheduling sets launches/token.** The `stream` dialect sequences dispatches with timepoints, removing per-kernel host waits the way a captured

launch graph does—worth roughly 1.26× on an H100 decode cell from host-overhead removal alone (Chen, 2026; NVIDIA, 2024).

- **A wide build matrix is not a wide decode matrix.** IREE targets CPU, CUDA, ROCm, Vulkan, and Metal, but the NPU/AIE path is experimental and a green target only means it lowers and runs (IREE / iree-org contributors, 2024; AMD, 2024b). Validate each target against decode metrics, not compilation success (AMD, 2024a).
- **Tuned configurations are per-HAL-target.** Cache-blocked CPU dispatch, fused CUDA kernels, and static AIE tiling are different strategies, so a configuration tuned for one HAL target does not transfer (SemiAnalysis, 2024; NVIDIA, 2022). Compile and profile at the batch-1 decode shape (Kwon *et al.*, 2023).

Practical decision rules. Four rules compress this chapter for review. First, set a dispatch-region target per decoder layer and treat the compile report as a gate, because region count predicts bytes/token before any benchmark runs (IREE / iree-org contributors, 2024; Chen, 2026). Second, benchmark through the VM or embedded runtime at batch-1, never individual regions, because the schedule between regions is part of the compiled product (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Third, fingerprint every artifact with the compiler version, flags, input hash, and benchmark so upgrades are reviewable diffs (Chen, 2020, 2026). Fourth, choose VM versus embedded mode and fat-binary versus per-SKU packaging from measured p99, not from build convenience (Kwon *et al.*, 2023; IREE / iree-org contributors, 2024). Teams that apply these rules treat IREE as a production compiler with a deployable schedule; teams that skip them treat it as a build system and discover the schedule only in fleet latency (Vellaisamy *et al.*, 2026; Chen, 2026).

1.7 Conclusion

IREE's contribution is to make whole-program scheduling a compiler artifact: by lowering through Flow, Stream, HAL, and VM it compiles host orchestration and device kernels together and packages them into one retargetable binary (IREE / iree-org contributors, 2024). For interactive serving that matters because it attacks launches/token at compile time and dispatch-region fusion attacks bytes/token—provided both are graded at the batch-1 decode operating point rather than on prefill-shaped regions (Vellaisamy *et al.*, 2026; Chen, 2026). That is the same

residency contract YiRage enforces as IR metadata across backends (Chen and YiRage Contributors, 2026). IREE also shows that breadth and depth are different axes: a wide HAL matrix still needs per-target residency proof, which sets up the next surface—Chapter **??** turns to Glow, a lighter-weight AI compiler that makes the opposite trade between generality and a focused lowering path (Goodfellow *et al.*, 2016).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 2, 4, 9, 11, 13, 15, 17

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 2, 4, 11, 16, 17

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 3, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 3, 13, 18

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 3, 6, 8, 11, 12, 13

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 4, 5, 11, 16, 18

IREE / iree-org contributors (2024). IREE: Intermediate representation execution environment — an MLIR-based end-to-end compiler and runtime. `https://iree.dev`. AOT MLIR compiler+runtime; Flow/Stream/HAL/VM dialects; dispatch-region fusion; `.vmfb` fat binary; targets llvm-cpu, CUDA, ROCm/HIP, Vulkan/SPIR-V, Metal, AMD AIE (experimental). 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 3, 7, 9, 10, 14, 15, 17

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 7, 11

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 2, 4, 8, 9, 11, 13, 17

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 5, 6, 7, 10, 14, 17

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 2, 6, 8, 14, 17

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Index

Hardware abstraction layer, 1

IREE, 1

MLIR, 1